

Advanced AI Medical Intelligence Platform

Project Report (Academic / Submission)

Product name: MedIntel

This report documents an end-to-end AI system for medical image analysis, disease prediction with deep learning, Grad-CAM explanations, LLM-assisted reporting, REST APIs, persistent prediction history, and a React web interface.

1. Project objective

Build a complete application capable of: analyzing medical images; predicting diseases using deep learning; explaining predictions with Grad-CAM; generating AI-assisted reports with an LLM; exposing REST APIs; storing history in a database; and deploying a user-friendly interface.

2. System architecture

The system uses a layered design: (1) React frontend for upload and visualization; (2) FastAPI backend for orchestration; (3) PyTorch ResNet18 classifier; (4) Grad-CAM explainer; (5) LLM/template report service; (6) SQLite persistence.

Layer	Technology
Frontend	React + TypeScript + Vite
Backend API	FastAPI + Uvicorn
Deep learning	PyTorch ResNet18 (fine-tuned)
XAI	Grad-CAM on ResNet layer4
LLM reports	OpenAI-compatible API + template fallback
Database	SQLite via SQLAlchemy
Deployment	Docker + Docker Compose

3. Deep learning model

A ResNet18 backbone pretrained on ImageNet is fine-tuned for multi-class medical image triage. Supported classes include brain (normal/tumor), chest (normal/pneumonia), bone fracture, abdomen, breast, eye/retina, skin, lower limb, and an UNSUPPORTED class used to reject unrelated uploads.

Training pipeline highlights:

- Data preparation scripts for chest, bone, brain, MedMNIST body regions
- Augmentation: flip, rotation, color jitter
- Class-weighted cross-entropy for imbalance
- Head-only warm-up then full fine-tuning
- Best validation checkpoint saved to backend/models/chest_xray_resnet18.pth

4. Explainable AI (Grad-CAM)

Grad-CAM computes class-discriminative localization maps from gradients flowing into the final convolutional block. The heatmap is overlaid on the original image and served via the media API so clinicians/users can see which regions influenced the prediction.

5. LLM-assisted medical reports

After prediction, the backend generates a structured report. If `OPENAI_API_KEY` is configured, an OpenAI-compatible chat completion is used. Otherwise a professional template report is produced. The UI also shows a disease-information card scoped to the predicted finding only.

6. REST API design

Method	Endpoint	Purpose
GET	/api/health	Health + model status
POST	/api/predict	Predict + Grad-CAM + report
GET	/api/history	List stored predictions
GET	/api/history/{id}	Fetch one prediction
POST	/api/reports	Regenerate report
DELETE	/api/history/{id}	Delete record

7. Database design

SQLite stores prediction history in table `predictions` with fields: `id`, `filename`, `original_filename`, `predicted_class`, `confidence`, `class_probabilities` (JSON text), `heatmap_path`, `report_text`, `model_mode`, `created_at`.

8. Web application

The React UI provides drag-and-drop upload, prediction display (finding + confidence only), Grad-CAM visualization, disease information for the predicted class, AI report text, and history browsing. Unsupported images are rejected with a clear message to upload a correct medical image.

9. Deployment

Dockerfiles exist for backend and frontend. `docker compose up --build` runs the stack. Local Windows scripts are also provided under `scripts/`. Public cloud deployment is optional for submission.

10. Evaluation criteria alignment

Criterion	Evidence in project
DL model performance	Fine-tuned ResNet18 + validation checkpoint
Code quality / structure	Modular backend & frontend packages
Explainable AI	Grad-CAM heatmaps in API/UI
LLM integration	OpenAI-compatible service + fallback
API development	FastAPI OpenAPI docs at /docs
Database design	SQLAlchemy prediction history schema
Web application	React medical triage UI
Documentation	README + this PDF report
Deployment	Dockerfile(s) + docker-compose.yml
System design / best practices	Layered architecture, reject unsupported inputs

11. Ethical disclaimer

MedIntel is an educational decision-support prototype. Predictions and reports are **not** clinical diagnoses and must be reviewed by licensed medical professionals before any clinical action.

12. Submission artifacts

- Complete source code in repository
- Trained model: backend/models/chest_xray_resnet18.pth
- README.md documentation
- This PDF project report
- requirements.txt (+ backend/requirements.txt)
- Dockerfile(s) and docker-compose.yml
- GitHub repository link (add after push)
- Live deployment link (optional)

— End of report —